Given:

To implement the K-Nearest Neighbors (KNN) algorithm using the Iris dataset, normalize the features, compare different values of K, evaluate the model using accuracy and confusion matrix, and visualize the decision boundaries.

DATASET: Click to Download Iris Dataset

KNN_Classification.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from matplotlib.colors import ListedColormap
df = pd.read_csv(r"..\DataSets\Iris.csv")
X = df.drop(["Id", "Species"], axis=1)
y = df["Species"]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
for k in [3, 5, 7, 9]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    print(f"\nK = {k}")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix")
    print(confusion_matrix(y_test, y_pred))
X = df[["PetalLengthCm", "PetalWidthCm"]].values
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
```

```
xx, yy = np.meshgrid(
   np.arange(x_min, x_max, 0.02),
   np.arange(y_min, y_max, 0.02)
)
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.figure(figsize=(8,6))
plt.contourf(
   xx,
   yy,
   Z,
   alpha=0.3,
   cmap=ListedColormap(("lightblue", "lightgreen", "lightpink"))
)
plt.scatter(
   X[:,0],
   X[:,1],
   c=y,
   cmap=ListedColormap(("blue", "green", "red")),
   edgecolor="k"
)
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("KNN Decision Boundary (K=5)")
plt.show()
```

Output:

K = 3
Accuracy: 0.9333333333333333
Confusion Matrix
[[10 0 0]
[0 10 0]
[0 2 8]]

K = 5
Accuracy: 0.9333333333333333
Confusion Matrix
[[10 0 0]
[0 10 0]
[0 2 8]]

K = 7
Accuracy: 0.9666666666666667
Confusion Matrix
[[10 0 0]
[0 10 0]
[0 1 9]]

K = 9
Accuracy: 0.9666666666666667
Confusion Matrix
[[10 0 0]
[0 10 0]
[0 1 9]]